



SDU International Summer School

Introduction to R

Day 4 - Packages and Data wrangling I

Paola Vazquez-Castillo

06 August 2026

Contents

1	Introduction	2
2	Packages	2
2.1	Installing packages	3
2.2	Using a package	4
2.3	Package documentation	4
3	A package in action	5
3.1	Data	5
4	Getting to know real data	7
4.1	Finding missing values	8
5	A second package	9
6	Other things worth knowing of	9
6.1	library("haven")	10
6.2	library("readxl")	10
6.3	library("clipr")	10
6.4	library(skimr)	10
6.5	library("janitor")	12
6.6	Masking	12
7	Data wrangling	13
8	Glossary of Functions	14
9	Exercises	15

1 Introduction

So far we have worked with the tools that come built into R. But a lot of R's usefulness comes from packages, collections of functions, data, and documentation that other people have written and shared. In this class we look at how to install and load packages, put one to work pulling real data straight from the World Bank, and then get a first taste of cleaning and reshaping that data with a couple of the tidyverse package tools.

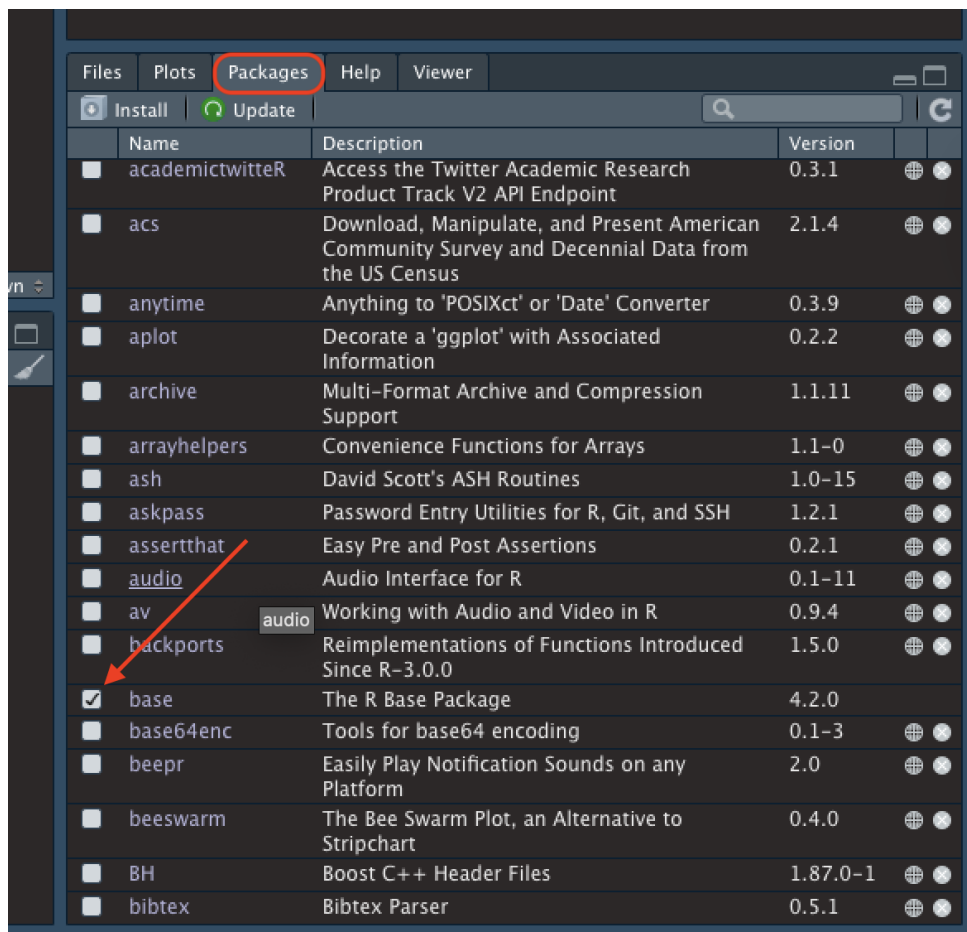
2 Packages

R packages contain functions, data, and documentation in a standardized collection format that can be installed by users of R, typically via CRAN (the Comprehensive R Archive Network). By default, R installs a set of basic packages during its installation. To see the currently loaded packages we use the function `search()`.

```
# See the packages currently loaded  
search()
```

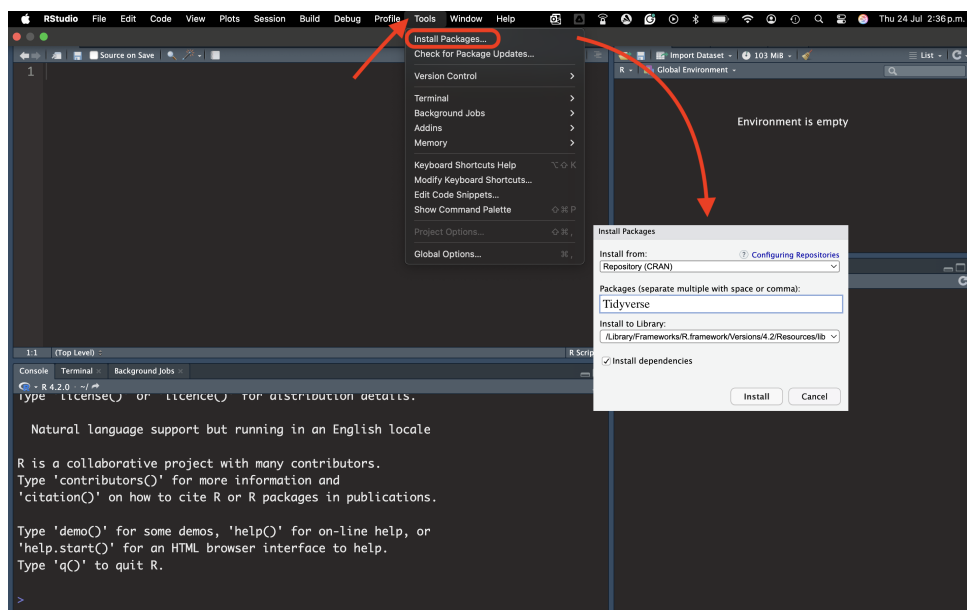
```
## [1] ".GlobalEnv"      "package:stats"    "package:graphics"  
## [4] "package:grDevices" "package:utils"    "package:datasets"  
## [7] "package:methods"  "Autoloads"        "package:base"
```

A way to see currently *installed* packages is on the bottom right panel in R studio. There you can see information on the general purpose of the package, the version of it that is downloaded and if it is currently loaded in the R session (check mark to the left).



2.1 Installing packages

More packages can be added when they are needed for some specific purpose. It is possible to install any package via the drop down menu by clicking in tools->install packages -> type the name of the package you want in the blank space -> click install.



NB! Once installed, a package stays on your computer until you delete it. So we only install packages once!

Another option is to install them directly via command with `install.packages("")`. Let's try to install a package called `tidyverse`. This is one of the most widely used packages in R, due to its clear grammar and usefulness for data processing, analysis, and visualization.

```
# Install an R package:
install.packages("tidyverse")
```

2.2 Using a package

R packages are stored under a directory called `library` in the R environment. That means that although you have permanently download it to your computer every session you need to say to R to use that package. We give that instruction to R by writing the function `library()` and the argument of the function is the name of the package.

```
# Load the package in R
library("tidyverse")
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.2.1      v readr      2.2.0
## v forcats    1.0.1      v stringr   1.6.0
## v ggplot2    4.0.3      v tibble    3.3.1
## v lubridate  1.9.5      v tidyr     1.3.2
## v purrr      1.2.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to b
```

NB: If you do not load the package, R will not be able to recognize the functions of it, so it is fundamental to load a package with the function `library()`

It is best practice to only load packages that you will use in the session. That helps to save memory and processing power.

2.3 Package documentation

Sometimes we need to see the documentation for a package to understand the functions included on it. We can access some information on the package by using the function `help()`. With that function we will see an overview of the package and lists all the functions/datasets it contains.

```
# overview of a package and its contents
help(tidyverse)
```

We can also search for help for a specific function to understand the arguments that need to be used or seeing some usage examples by using a question mark followed by the name of a package. If you are unsure of the function to be used, you can use two question marks followed by the name of the function which will show all the help documentation containing the search term.

```
# Help for one specific function
?select
```

```
# Search for all help containing this term
??tidyverse
```

If we want to read more documentation of the package (not necessarily only its functions), we can use the function `browseVignettes()`.

```
browseVignettes("tidyverse")
```

```
## starting httpd help server ... done
```

Finally, it is always good practice when using packages for the first time to searching the package online and explore more about how other people use that package .

3 A package in action

3.1 Data

So far, we have whether created our data frames or using dataframes previously stored in our computers, for example using the function `read.csv()`. Today we will test the power of one of the packages in R by using the package `wbstats`. This is a package that connects directly to the World Bank API and provides access to updated statistics on diverse topics.

```
# install the package
install.packages("wbstats")
```

After installing the package, there are two things that one needs to do if wanting to use the package in the R session. First, LOAD the package using `library()`. Second, view the help guide to see more about the functions and so on.

```
# LOAD the package in the current R session
library(wbstats)
# Since we are new to the package
help(wbstats)
```

The documentation of the packages mentions to use the function `wb_search()` in order to find the code of the indicators that we are interested in. Let's, for example search for Life Expectancy, Gross Domestic Product per capita (GDP), and population size.

```
# Searching for Life expectancy at birth, total (years) -> SP.DYN.LE00.IN
wb_search("Life Expectancy")
```

```
## # A tibble: 42 x 3
##   indicator_id      indicator      indicator_desc
##   <chr>            <chr>            <chr>
## 1 GCI.4THPILLAR.XQ 4th pillar: Health and primary education This indicato~
## 2 SE.SCH.LIFE      School life expectancy, primary to tertiary~ Number of yea~
## 3 SE.SCH.LIFE.FE   School life expectancy, primary to tertiary~ Number of yea~
## 4 SE.SCH.LIFE.MA   School life expectancy, primary to tertiary~ Number of yea~
## 5 SP.DYN.LE00.FE.IN Life expectancy at birth, female (years)    Life expectan~
## 6 SP.DYN.LE00.IN   Life expectancy at birth, total (years)      Life expectan~
## 7 SP.DYN.LE00.MA.IN Life expectancy at birth, male (years)       Life expectan~
## 8 SP.DYN.LE60.FE.IN Life expectancy at age 60, female (years)    Life expectan~
## 9 SP.DYN.LE60.MA.IN Life expectancy at age 60, male (years)      Life expectan~
```

```
## 10 UIS.SLE.02          School life expectancy, pre-primary, both s~ <NA>
## # i 32 more rows
```

```
# Searching for GDP per capita (current US$) -> NY.GDP.PCAP.CD
wb_search("GDP per capita")
# Searching for Population, total -> SP.POP.TOTL
wb_search("Population, total")
```

If we were interested in a different variable, we should browse in the World Bank Open Data catalog (<https://data.worldbank.org>) and search for the figure we are interested in. Then use `wb_search()` to find its corresponding code.

In order to retrieve the data to be able to use it, we must use the function `wb_data()`. So let's create a dataframe (as seen in class 2) that contains all the information needed. First, it is needed to create a dataframe containing the indicators to retrieve from the world bank.

```
my_indicators <- c(
  life_exp = "SP.DYN.LE00.IN",
  pop = "SP.POP.TOTL",
  gdp="NY.GDP.MKTP.CD",
  gdp_capita = "NY.GDP.PCAP.CD"
)
```

Once the indicators are selected, we can retrieve them from the World Bank using the function `wb_data()`. The function has arguments to select different things, among them the first year we want the observations and the last one.

```
# This is the information I get from WB
wb_data(my_indicators, start_date = 2014, end_date = 2024)
```

```
## # A tibble: 2,387 x 8
##   iso2c iso3c country  date      gdp gdp_capita life_exp  pop
##   <chr> <chr> <chr>   <dbl>    <dbl>    <dbl>    <dbl>
## 1 AW    ABW    Aruba   2014 2790849721.    26130.    75.3 106807
## 2 AW    ABW    Aruba   2015 2962906704.    27458.    75.4 107906
## 3 AW    ABW    Aruba   2016 2983637430.    27442.    75.5 108727
## 4 AW    ABW    Aruba   2017 3092427933.    28440.    75.6 108735
## 5 AW    ABW    Aruba   2018 3276187709.    30082.    75.9 108908
## 6 AW    ABW    Aruba   2019 3347561739.    30654.    76.0 109203
## 7 AW    ABW    Aruba   2020 2461056053.    22664.    75.4 108587
## 8 AW    ABW    Aruba   2021 2889305034.    26827.    73.7 107700
## 9 AW    ABW    Aruba   2022 3326671315.    31001.    76.2 107310
## 10 AW   ABW    Aruba   2023 3746573413.    34898.    76.4 107359
## # i 2,377 more rows
```

```
# Now I will store it into a data.frame
data <- data.frame(wb_data(my_indicators, start_date = 2014, end_date = 2024))
# Testing if it is a data frame
class(data)
```

```
## [1] "data.frame"
```

We have stored now the information in the dataframe called `data`. Lets take a look to the structure of the data

```
# Visualizing first entries of the dataframe
```

```
head(data)
```

```
##   iso2c iso3c country date      gdp gdp_capita life_exp  pop
## 1    AW  ABW  Aruba 2014 2790849721  26129.84  75.261 106807
## 2    AW  ABW  Aruba 2015 2962906704  27458.22  75.405 107906
## 3    AW  ABW  Aruba 2016 2983637430  27441.55  75.540 108727
## 4    AW  ABW  Aruba 2017 3092427933  28440.04  75.620 108735
## 5    AW  ABW  Aruba 2018 3276187709  30082.16  75.880 108908
## 6    AW  ABW  Aruba 2019 3347561739  30654.49  76.019 109203
```

```
#Visualizing last entries of the dataframe
```

```
tail(data)
```

```
##   iso2c iso3c country date      gdp gdp_capita life_exp  pop
## 2382   ZW  ZWE Zimbabwe 2019 33360632553  2184.522  61.060 15271368
## 2383   ZW  ZWE Zimbabwe 2020 31979753645  2059.637  61.530 15526888
## 2384   ZW  ZWE Zimbabwe 2021 41287852524  2613.617  60.135 15797210
## 2385   ZW  ZWE Zimbabwe 2022 40757585309  2536.402  62.360 16069056
## 2386   ZW  ZWE Zimbabwe 2023 35871781303  2195.225  62.775 16340822
## 2387   ZW  ZWE Zimbabwe 2024 41521975830  2496.155  63.064 16634373
```

4 Getting to know real data

Data pulled from the real world is rarely in a nice and clean shape. Before doing any analysis or model with it, we must inspect it.

```
#structure and quick overview
```

```
summary(data)
```

```
##           iso2c           iso3c           country           date
## Length      :2387   Length      :2387   Length      :2387   Min.       :2014
## N.unique    : 217   N.unique    : 217   N.unique    : 217   1st Qu.  :2016
## N.blank     :   0   N.blank     :   0   N.blank     :   0   Median    :2019
## Min.nchar   :   2   Min.nchar   :   3   Min.nchar   :   4   Mean      :2019
## Max.nchar   :   2   Max.nchar   :   3   Max.nchar   :  30   3rd Qu.  :2022
##                                     Max.       :2024
##
##           gdp           gdp_capita           life_exp           pop
## Min.       :3.676e+07   Min.       :  216.2   Min.       :18.82   Min.       :9.646e+03
## 1st Qu.    :6.813e+09   1st Qu.    : 2515.1   1st Qu.    :67.08   1st Qu.    :7.782e+05
## Median     :2.910e+10   Median     : 7051.2   Median     :73.75   Median     :6.427e+06
## Mean       :4.301e+11   Mean       :19387.4   Mean       :72.72   Mean       :3.566e+07
## 3rd Qu.    :1.943e+11   3rd Qu.    :25353.1   3rd Qu.    :78.60   3rd Qu.    :2.488e+07
## Max.       :2.930e+13   Max.       :288001.6   Max.       :86.50   Max.       :1.451e+09
## NAs        :85        NAs        :85
```

```
# how many rows and columns do we have?
```

```
dim(data)
```

```
## [1] 2387      8
```

```
# What years does the data cover
unique(data$date)
```

```
## [1] 2014 2015 2016 2017 2018 2019 2020 2021 2022 2023 2024
```

```
# How many unique countries do we have?
length(unique(data$country))
```

```
## [1] 217
```

Another very useful function for inspecting data at once is `glimpse()`.

```
glimpse(data)
```

```
## Rows: 2,387
## Columns: 8
## $ iso2c      <chr> "AW", "AW", "AW", "AW", "AW", "AW", "AW", "AW", "AW", "AW", ~
## $ iso3c      <chr> "ABW", "ABW", "ABW", "ABW", "ABW", "ABW", "ABW", "ABW", "AB~
## $ country    <chr> "Aruba", "Aruba", "Aruba", "Aruba", "Aruba", "Aruba", "Arub~
## $ date       <dbl> 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, ~
## $ gdp        <dbl> 2790849721, 2962906704, 2983637430, 3092427933, 3276187709, ~
## $ gdp_capita <dbl> 26129.8391, 27458.2202, 27441.5502, 28440.0417, 30082.1584, ~
## $ life_exp   <dbl> 75.261, 75.405, 75.540, 75.620, 75.880, 76.019, 75.406, 73.~
## $ pop        <dbl> 106807, 107906, 108727, 108735, 108908, 109203, 108587, 107~
```

4.1 Finding missing values

Real data typically is not complete. That means that it has gaps. For example, a country may not report every indicator every year, and those gaps show up as NA. We saw before what is an NA, with the function `is.na()` we can find them. The function will return TRUE wherever a value is missing (it tests the condition *is the value NA*?). Using `is.na()` is different from `value==NA` because the first one will return a TRUE or FALSE while the latter one will always return NA, so when doing a comparison we use `is.na()` instead.

```
# Creating a vector
x<-c(1,2,3,4,NA,6)
# Testing if the values are NA
is.na(x)
```

```
## [1] FALSE FALSE FALSE FALSE TRUE FALSE
```

Sometimes, when we have bigger datasets and vectors, rather than seeing the results of the function `is.na()`, we would prefer to see a summary of the values that are NA.

```
# Total number of missing values in the whole dataframe
sum(is.na(data))
```

```
## [1] 170
```

```
# Missing values per column (more usable)
colSums(is.na(data))
```

```
##      iso2c      iso3c      country      date      gdp gdp_capita      life_exp
##         0         0         0         0        85         85         0
##      pop
```



```
##           0
```

`summary()` also reports an NA's count for each numeric column, so the two together tell you where the data gaps are.

5 A second package

Our data has country names but no region. The `countrycode` package translates between country coding schemes and can also roll countries up to continents. This is another example of the same idea: a package handing us a new capability.

```
install.packages("countrycode")
```

```
library(countrycode)
```

```
help("countrycode")
```

```
# Add a continent column (creating a new variable, as in Day 2)
```

```
data$continent <- countrycode(  
  sourcevar = data$iso3c,  
  origin    = "iso3c",  
  destination = "continent"  
)
```

```
## Warning: Some values were not matched unambiguously: CHI, XKX
```

```
## To fix unmatched values, please use the 'custom_match' argument. If you think the default
```

```
# Verify our continents
```

```
unique(data$continent)
```

```
## [1] "Americas" "Asia"      "Africa"    "Europe"    "Oceania"  NA
```

```
# How many NAs do we have
```

```
sum(is.na(data$continent))
```

```
## [1] 22
```

```
# Which are the countries that have NA values?
```

```
data$country[is.na(data$continent)]
```

```
## [1] "Channel Islands" "Channel Islands" "Channel Islands" "Channel Islands"  
## [5] "Channel Islands" "Channel Islands" "Channel Islands" "Channel Islands"  
## [9] "Channel Islands" "Channel Islands" "Channel Islands" "Kosovo"  
## [13] "Kosovo"          "Kosovo"          "Kosovo"          "Kosovo"  
## [17] "Kosovo"          "Kosovo"          "Kosovo"          "Kosovo"  
## [21] "Kosovo"          "Kosovo"
```

6 Other things worth knowing of

R has a very extensive list of packages (24409 up to today!) ready to use to gather, analyze, and visualize data. We cannot talk about all of them in one class but this are some of the other packages that might be useful for you.

6.1 library("haven")

When your data comes from another statistical program, the `haven` package reads it directly. Survey datasets are often shared as SPSS (`.sav`), Stata (`.dta`), or SAS files, and `haven` turns them into an ordinary data frame.

```
install.packages("haven")
library(haven)
# read_sav() for SPSS, read_dta() for Stata, read_sas() for SAS
# my_data <- read_sav("survey.sav")
```

6.2 library("readxl")

Excel spreadsheets are everywhere too. The `readxl` package reads `.xls` and `.xlsx` files without needing Excel installed.

```
install.packages("readxl")
library(readxl)
# read_excel() reads the first sheet by default but you can specify a different sheet
# my_data <- read_excel("data.xlsx")
```

6.3 library("clipr")

To copy an R object to the clipboard so you can paste it into a spreadsheet, an email, or a chat.

```
install.packages("clipr")

library(clipr)

## Welcome to clipr. See ?write_clip for advisories on writing to the clipboard in R.
write_clip(data)
```

6.4 library("skimr")

The library `skimr` is designed to provide summary statistics about data frames and variables in data frames. One of the most used functions is `skim()`, that gives a more polished overview of the variables compared to, for example, `summary()`.

```
install.packages("skimr")

library("skimr")
# How does summary look?
summary(data)
```

```
##           iso2c           iso3c           country           date
## Length   :2387   Length   :2387   Length   :2387   Min.     :2014
## N.unique  : 217   N.unique  : 217   N.unique  : 217   1st Qu.:2016
## N.blank   :    0   N.blank   :    0   N.blank   :    0   Median  :2019
## Min.nchar :    2   Min.nchar :    3   Min.nchar :    4   Mean    :2019
## Max.nchar :    2   Max.nchar :    3   Max.nchar :   30   3rd Qu.:2022
##                                     Max.    :2024
##
##           gdp           gdp_capita           life_exp           pop
```

```
## Min.      :3.676e+07   Min.      : 216.2   Min.      :18.82   Min.      :9.646e+03
## 1st Qu.:6.813e+09   1st Qu.: 2515.1   1st Qu.:67.08   1st Qu.:7.782e+05
## Median :2.910e+10   Median : 7051.2   Median :73.75   Median :6.427e+06
## Mean    :4.301e+11   Mean    : 19387.4   Mean     :72.72   Mean     :3.566e+07
## 3rd Qu.:1.943e+11   3rd Qu.: 25353.1   3rd Qu.:78.60   3rd Qu.:2.488e+07
## Max.     :2.930e+13   Max.     :288001.6   Max.      :86.50   Max.      :1.451e+09
## NAs      :85         NAs      :85
## continent
## Length    :2387
## N.unique   : 5
## N.blank    : 0
## Min.nchar  : 4
## Max.nchar  : 8
## NAs        : 22
##
```

Using the package function:

```
skim(data)
```

Table 1: Data summary

Name	data
Number of rows	2387
Number of columns	9
Column type frequency:	
character	4
numeric	5
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
iso2c	0	1.00	2	2	0	217	0
iso3c	0	1.00	3	3	0	217	0
country	0	1.00	4	30	0	217	0
continent	22	0.99	4	8	0	5	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100
date	0	1.00	2.019000e-20	1.000000e-20	2.014000e-20	2.016000e-20	2.019000e-20	2.022000e-20	2.024000e+03
gdp	85	0.96	4.301436e+11	1.954899e+12	2.529067e+11	8.12629e+11	2.910346e+11	1.012808e+12	2.1229801e+13
gdp_capita	85	0.96	1.938735e+08	2.896677e+08	2.1623e+07	2.515150e+07	7.051160e+07	2.535307e+08	2.880016e+05
life_exp	0	1.00	7.272000e+01	1.040000e+01	18.82	6.708000e+01	7.375000e+01	7.860000e+01	8.650000e+01
pop	0	1.00	3.565883e+08	1.385594e+09	86.00	7.781780e+01	6.427382e+01	1.688482e+02	1.470936e+09

6.5 library("janitor")

This library is a toolkit for cleaning and exploring messy data in an easy way. A key function is `clean_names()` to fix inconsistent column naming but the package contains many other useful tools.

```
install.packages("janitor")

library(janitor)

##
## Attaching package: 'janitor'
##
## The following objects are masked from 'package:stats':
##
##      chisq.test, fisher.test
##
# Creating a data frame with bad names
badDF<-data.frame("ColumnA"=c(1,2,3),
                  "columnB"=c(1,2,3),
                  "COL4$"=c(1,2,3))

# Cleaning the names of the data
cleaned_DF<-clean_names(badDF)
names(cleaned_DF)

## [1] "column_a" "column_b" "col4"
```

6.6 Masking

Since packages are created by different authors, it is common that sometimes multiple packages have the same name for different functions. This is called *masking*. For example, if you use `dplyr` and `stats` have the same name for a function called `lag`. In `dplyr` it displays the previous observation in a vector, i.e. the position `i-1`. In `stats`, `lag` is mostly used for time-series objects.

```
library(dplyr)
lag(1:5)

## [1] NA  1  2  3  4

library(stats)
lag(1:5)

## [1] NA  1  2  3  4
```

Masking just means two packages have a function with the same name, and R uses the one from the package loaded most recently.

There is several solutions to this but there is two easy ones. One, if you know this is happening, you can load the package that contains the version of the function you want to use at the end. In that way you ensure that the function from that package is not masked. The second solution is to say to R explicitly you want to use a function from a given package, that is done by writing: the package name, `::` double semicolons), and then the name of the function (`packagename::function()`).

```
stats::lag(1:5)
```

```
## [1] 1 2 3 4 5
## attr("tsp")
## [1] 0 4 1
```

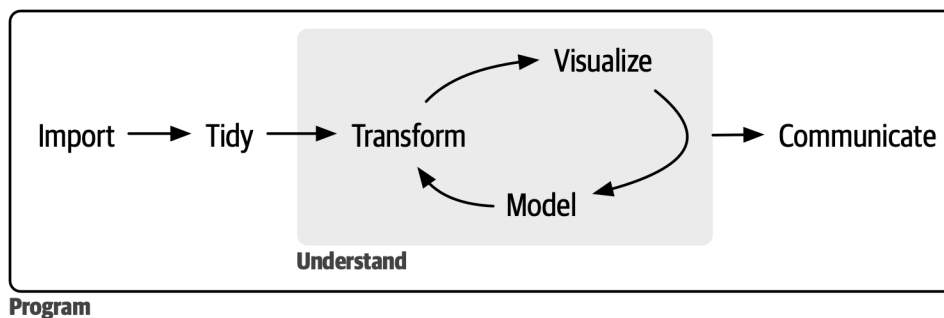
```
dplyr::lag(1:5)
```

```
## [1] NA 1 2 3 4
```

Everyone can write a package. If you think you have a cool idea or you developed a programming solution in a creative way that is not publicly available yet, then you can create your own package!

7 Data wrangling

Analyzing data is a process that has several steps. After importing, first step is *data wrangling*. Data wrangling or manipulation is the process in which: 1) we clean (tidy) the data and 2) transform it to have the variables we need so that we can then analyze it.



Source: Wickham, Çetinkaya-Rundel & Golemund (2026)

One of the biggest packages of R is the **tidyverse**. The **tidyverse** is a package well known for its collection of functions suited for different steps data wrangling.

We will talk more about this package in the coming days but in the mean time, we will borrow from it three verbs (tidyverse term for functions) to transform data. The first one is **filter()**. **filter()** changes which *rows* are present in the data given a defined condition without changing their order.

For example, using our data set: **data**, we will transform it to a subset that contains data only for Denmark, Norway, and Finland in the year 2020.

```
# Select the observations in year 2020 for Denmark countries
filter(data, date==2020 & country %in% c("Denmark", "Norway", "Finland"))
```

```
##   iso2c iso3c country date      gdp gdp_capita life_exp      pop continent
## 1    DK   DNK  Denmark 2020 355631021932    60985.49 81.60244 5831404    Europe
## 2    FI   FIN  Finland 2020 270000311606    48828.68 81.93171 5529543    Europe
## 3    NO   NOR   Norway 2020 382252517922    71057.59 83.20976 5379475    Europe
```

filter() executes the operation and then prints it but it does not modify the original dataset. To do that we need to save the results in an object. Most of the tidyverse functions have the same behavior.

```
data_norsk_Y2020<-filter(data, date==2020 & country %in% c("Denmark", "Norway", "Finland"))
data_norsk_Y2020
```

```
##   iso2c iso3c country date      gdp gdp_capita life_exp      pop continent
## 1    DK   DNK Denmark 2020 355631021932  60985.49 81.60244 5831404    Europe
## 2    FI   FIN Finland 2020 270000311606  48828.68 81.93171 5529543    Europe
## 3    NO   NOR Norway  2020 382252517922  71057.59 83.20976 5379475    Europe
```

The second verb is `select()`. With `select` we reduce the columns from a dataframe and select just the ones we want.

As an example we will select only the columns: `continent`, `country`, `gdp`, `pop`, and `gdp_capita`.

```
# Select continent, country, gdp, pop, and gdp_capita columns
data_small<-select(data_norsk_Y2020, continent, country, gdp, pop, gdp_capita)
data_small
```

```
##   continent country      gdp      pop gdp_capita
## 1    Europe Denmark 355631021932 5831404   60985.49
## 2    Europe Finland 270000311606 5529543   48828.68
## 3    Europe Norway  382252517922 5379475   71057.59
```

Notice that `select` arranged the columns in the same order I selected them in.

The third function we will explore today is `mutate()`. `mutate()` is the `tidyverse` function that allow us to create new variables (columns) while keeping the previous ones. As an example we will create a new variable that takes the `gdp` and divides it by population.

```
data_new<-mutate(data_small,new_var=gdp/pop, log_gdp=log(gdp))
data_new
```

```
##   continent country      gdp      pop gdp_capita new_var  log_gdp
## 1    Europe Denmark 355631021932 5831404   60985.49 60985.49 26.59716
## 2    Europe Finland 270000311606 5529543   48828.68 48828.68 26.32169
## 3    Europe Norway  382252517922 5379475   71057.59 71057.59 26.66935
```

These three functions: `filter()`, `select()`, and `mutate()` are part of the data transformation functions available in the `tidyverse`. The next days we will learn more about this functions and we will extend to tidying functions and visualization.

8 Glossary of Functions

Function	What it does
<code>search()</code>	Lists the packages loaded in the current session
<code>install.packages()</code>	Installs a package onto your computer (once)
<code>library()</code>	Loads an installed package into the session
<code>help()</code> , <code>?</code>	Open the documentation for a package or function
<code>??</code>	Searches all help pages for a term
<code>browseVignettes()</code>	Opens the longer guides for a package
<code>wb_search()</code>	Finds World Bank indicator codes by keyword

Function	What it does
<code>wb_data()</code>	Downloads World Bank indicators as a data frame
<code>class()</code>	Reports the type of an object
<code>head()</code> , <code>tail()</code>	Show the first or last rows of a data frame
<code>summary()</code>	Per-column overview, including NA counts
<code>dim()</code>	Number of rows and columns
<code>unique()</code>	Returns the distinct values of a variable
<code>length()</code>	Number of elements in a vector
<code>glimpse()</code>	Compact column-by-column view of a data frame
<code>is.na()</code>	Returns TRUE wherever a value is missing
<code>sum()</code> , <code>colSums()</code>	Total, or per-column, count of NAs when wrapped around <code>is.na()</code>
<code>countrycode()</code>	Translates country codes and adds regions or continents
<code>read_sav()</code> , <code>read_dta()</code> , <code>read_sas()</code>	Read SPSS, Stata, and SAS files (haven)
<code>read_excel()</code>	Reads Excel files (readxl)
<code>write_clip()</code>	Copies an R object to the clipboard (clipr)
<code>skim()</code>	Polished summary of a data frame (skimr)
<code>clean_names()</code>	Standardizes messy column names (janitor)
<code>::</code>	Calls a function from a specific package by name
<code>select()</code>	Keeps chosen columns
<code>filter()</code>	Keeps rows that meet a condition
<code>mutate()</code>	Define new columns

9 Exercises

1. Load `wbstats` and use `wb_search()` to find the indicator “Mortality rate, infant”. Identify its indicator code, then retrieve it with `wb_data()` for the years 2000 to 2020 and store it in a data frame called `infant`.
2. Get to know your data. Use `head()`, `summary()`, and `glimpse()` to look it over, then `colSums(is.na())` to count the missing values in each column. Write one sentence saying what do you see from the NAs in the columns?
3. Add a `continent` column to `infant` with `countrycode` (from `iso3c` to `continent`). Use `is.na()` to list the countries that did not match, as we saw with the Channel Islands and Kosovo.
4. Use `select()` to keep only `country`, `date`, `continent`, and the infant mortality column. Then use `filter()` to keep the African countries in 2015. Which country in that subset has the highest infant mortality?
5. Using `filter()`, make a subset for Denmark across all years. Then, with the `plot()` function from Day 1, plot infant mortality (y) against `date` (x) as a line (`type = "l"`). Give it a title and axis labels. What do you see?